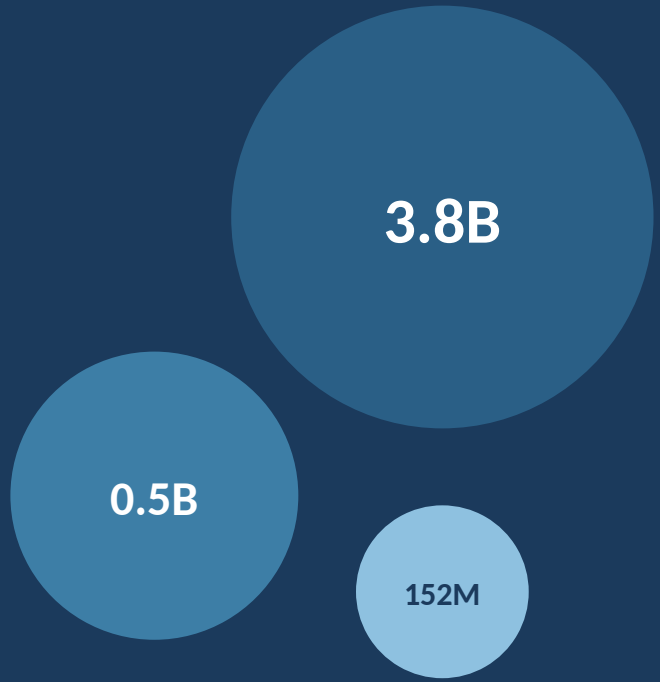
Load Balancer para Modelos de IA

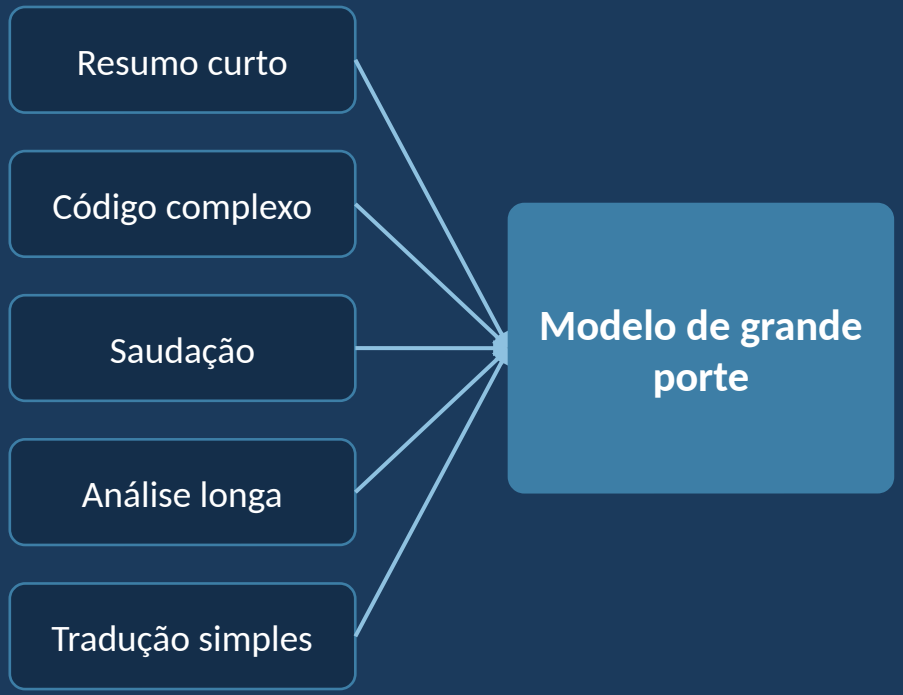
Roteamento de requisições por complexidade entre modelos de IA de diferentes portes



PUC Minas · Trabalho Interdisciplinar V
Artur Fernandes Braga de Menezes · Artur Rizzi Martinho · Felipe Lacerda Tertuliano · Gabriel Alves da Silva Diógenes · Rafael Mortimer Colares

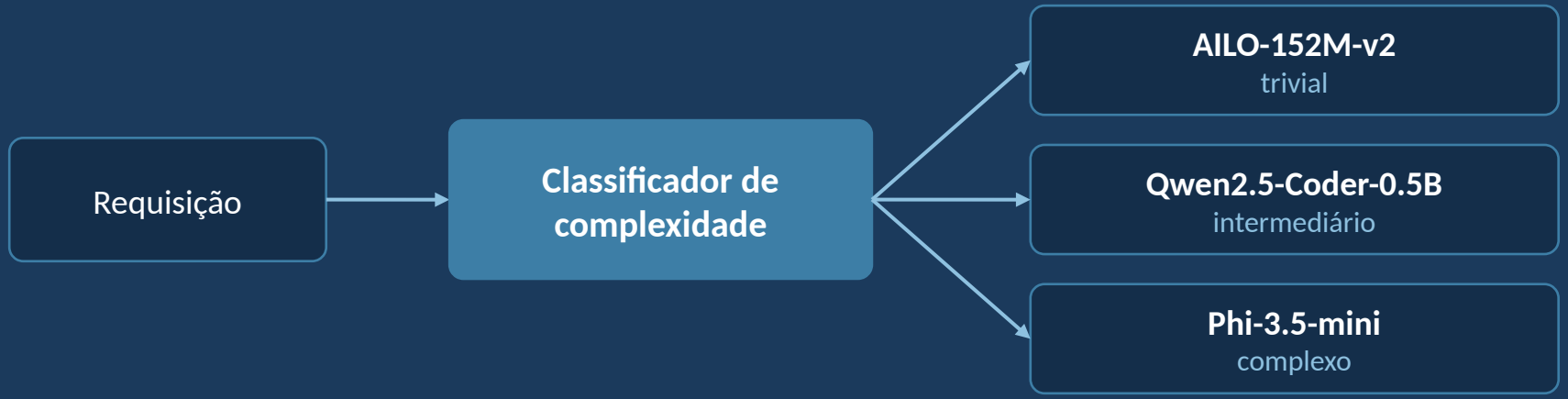
Um modelo só para tudo

- Toda requisição, simples ou complexa, vai para o mesmo modelo de grande porte
- Uso ineficiente de GPU/CPU em tarefas simples
- Maior latência e custo de inferência
- Demanda heterogênea, capacidade subutilizada



Rotear pela complexidade do pedido

- Um load balancer que classifica cada prompt e o direciona ao modelo mais adequado
- Evita usar um modelo grande quando um modelo leve resolve



Serviço via Ollama · orquestração em Python (FastAPI + httpx)

O que o trabalho pretende mostrar

OBJETIVO GERAL

Desenvolver um load balancer que direcione requisições entre modelos de IA de portes distintos, conforme sua complexidade

1. Implementar um classificador de prompts
2. Reduzir latência média e custo de inferência
3. Comparar baseline (um único modelo) x proposta
4. Medir uso de RAM e GPU por instância

O que já existe — e o que falta

Kossmann et al. (2024)	SAL: balanceia pela carga de fila e memória do servidor
Jain et al. (2025)	Prevê o tamanho da resposta entre réplicas do mesmo modelo
Sun et al. (2024)	Llumnix: reagenda requisições em execução entre instâncias
Xia et al. (2025)	SkyWalker: balanceamento entre regiões geográficas
Yuan et al. (2026)	DualMap: concilia afinidade de cache com balanceamento de carga

Lacuna: nenhum classifica a complexidade do prompt para escolher entre modelos de portes diferentes.

Baseline x proposta, em medição

BASELINE

Todas as requisições vão para o modelo de maior porte

PROPOSTA

O LoadBalancer classifica o prompt e encaminha ao modelo mais adequado

MÉTRICAS MEDIDAS

- **Latência média, P95 e P99**
- **Uso de RAM e GPU por máquina**
- **Identificação de outliers**

Estado atual — Sprint 2

CONCLUÍDO

✅ Load Balancer completo (FastAPI + httpx + fallback)

✅ Classificador por regras (5 regras + 2 casos ambíguos)

✅ Script de experimentos (baseline vs proposta, contrabalançado)

✅ 40 prompts rotulados (14 simples, 13 código, 13 complexos)

✅ Testes automatizados (unitários + integração)

✅ Documentação completa (README + .env.example + API)

EM ANDAMENTO

⚠️ Resultados preliminares disponíveis — medição completa na semana 5

⚠️ Análise estatística completa (P95, P99, outliers) — prevista para a semana 5

Dificuldades enfrentadas

1

Registro manual dos modelos

Cada modelo é adicionado à mão e associado a uma classe no classificador. Adicionar um novo exige editar o código e reiniciar.

2

Conexão com máquinas externas

Expor o Ollama fora da rede local: endereços e portas, autenticação e variação de latência de rede.

3

Medição de latência sem ruído

Garantir que a latência reflita o roteamento, e não carga da máquina, ordem dos prompts ou cache de KV.

O que já foi feito para contornar

Classificador determinístico por regras

Decisão estável entre rodadas, explicável no log, sem variação introduzida por aprendizado.

Fallback automático para o Phi-3.5-mini

Se o modelo escolhido não responde, o prompt vai ao modelo de reserva e o redirecionamento é registrado.

Experimento contrabalançado

Metade dos prompts vai primeiro ao baseline, metade à proposta; nenhum prompt roda duas vezes seguidas; aquecimento antes da medição.

Medição de ponta a ponta

Latência medida no cliente com time.perf_counter, independente do servidor.

O que fica para as próximas semanas

Semana 4	**Refinamento do classificador** Novas categorias, acurácia sobre o conjunto rotulado, monitoramento
Semana 5	**Análise estatística completa** Mais testes, outliers e comparação formal entre baseline e proposta
Semanas 6–8	**Otimização e escrita** Testes de roteamento, resultados do artigo e trabalhos relacionados
Sprint 3	**Pitch e submissão** Gravação do pitch comercial, revisão final do artigo e documentação

Cronograma do semestre

SEMANAS 1–2

Ambiente e classificador — Setup do Ollama e 1ª versão do LoadBalancer

SPRINT 2 · SEM. 3–4

Implementação e métricas — Refinamento do classificador e monitoramento

SEMANAS 5–6

Experimentos — Testes comparativos e otimização

SEMANAS 7–8

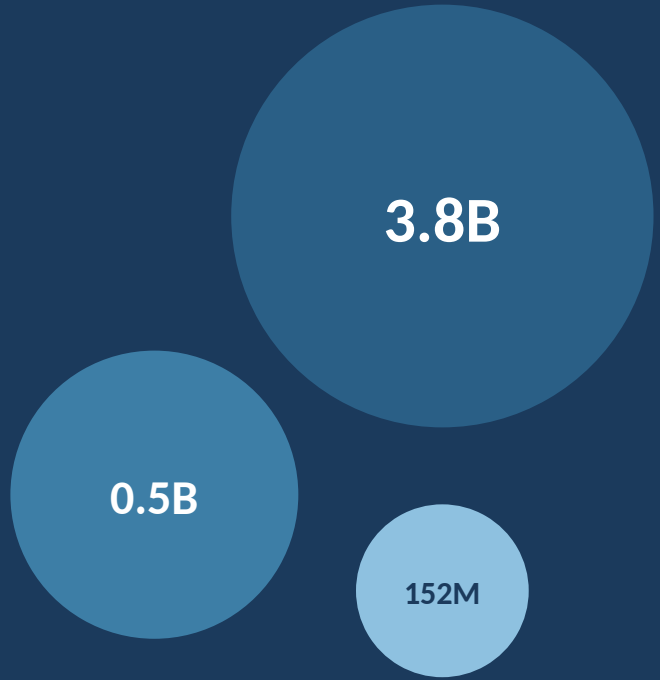
Análise e escrita — Comparação com trabalhos relacionados

SPRINT 3 · SEM. 9–10

Entrega final — Pitch comercial e submissão do artigo

Obrigado

Perguntas e discussão



PUC Minas · Trabalho Interdisciplinar V
Artur Fernandes Braga de Menezes · Artur Rizzi Martinho · Felipe Lacerda Tertuliano · Gabriel Alves da Silva Diógenes · Rafael Mortimer Colares